Silsila

How the whole system works

A study guide in simple language:
every class, how the pages are built, how login and passwords work,
and how all the parts talk to each other.

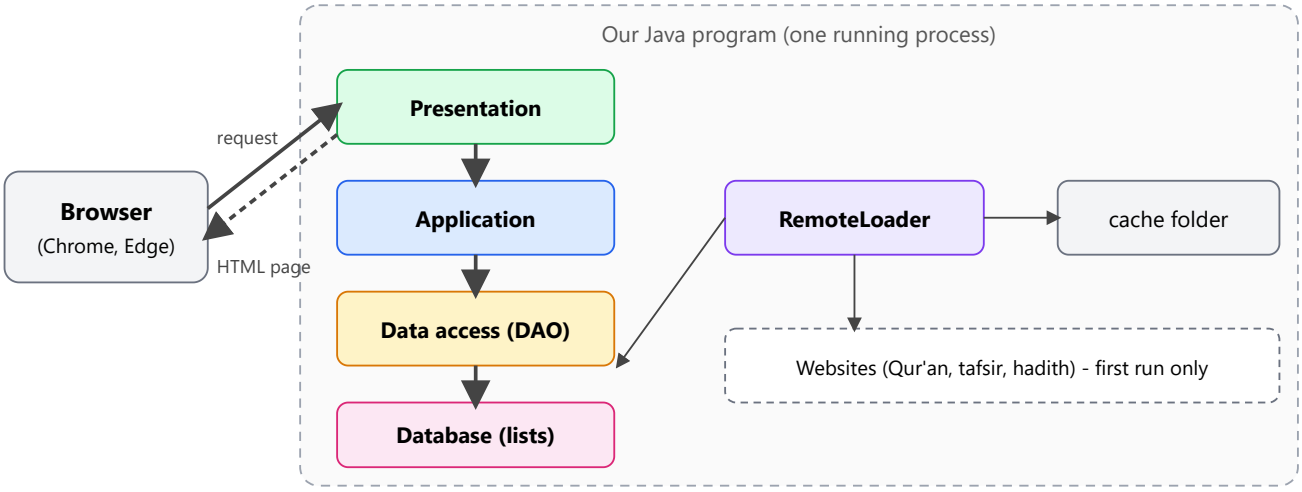
Contents

1. The big picture
2. Words you must know
3. The folders and how the program starts
4. How the browser and Java talk (and how HTML is joined with Java)
5. The domain classes
6. The Database and the content files
7. The DAO classes
8. The services (the logic)
9. The presentation classes (pages, forms, login)
10. Loading verses and hadith from websites
11. Full walk-throughs of the main actions
12. Security: what protects what
13. Why we designed it this way
14. Limits (be honest about these)
15. Questions the doctor may ask, with answers
16. One-line cheat sheet of every class

1. The big picture

What the project is. A website for students of fiqh. A student searches for a fiqh issue (for example "Is a wali required for the marriage contract?") and sees the whole chain of evidence: the verse, its tafsir, the hadith, what each of the four madhabs says, and the modern law. Every item shows where it comes from (its citation). Contributors can propose new connections, and reviewers approve or reject them. Students only see approved connections.

What it is made of. Plain Java 22. No framework (no Spring). The web server is the one that comes with Java (`com.sun.net.httpserver.HttpServer`). The pages are HTML text that our Java code builds. The data lives in Java lists in memory. The text of verses, tafsir and hadith is downloaded from public websites and saved in a `cache` folder.



> **One sentence to remember:** the browser sends a request; *Presentation* reads it and asks *Application*; *Application* asks *Data access*; *Data access* reads the *Database*. The answer goes back up the same road and ends as an HTML page.

2. Words you must know

Word	Simple meaning
Server	A program that waits for requests from browsers and answers them. Ours waits on port 8080.
Port	A door number on your computer. `localhost:8080` means "this computer, door 8080".
HTTP request	The message the browser sends: "give me this page" (with a method like GET or POST, an address, and some extra information called headers).
HTTP response	The message the server sends back: a status code, headers, and the page.
GET	A request that only asks to see something (opening a page, searching). The data goes in the address: `/search?q=wali`.

POST	A request that sends data to change something (login, propose, approve). The data goes in the body of the request, not in the address.
Status code	A number: **200** OK, **303** go to another address (redirect), **403** forbidden (you are not allowed), **404** not found.
HTML	The text language of web pages (`<h1>Title</h1>`). The browser turns it into what you see.
CSS	The styling rules (colours, spacing). Ours are inside a `<style>` tag in every page.
Query string	The part of the address after `?`, like `q=wali` or `id=4`.
Form	An HTML box with fields and a button. When you press the button the browser sends the fields.
Cookie	A small text the server gives to the browser. The browser sends it back on every later request. We use one cookie called `session`.
Session	The server's memory of "this cookie belongs to this logged-in user".
JSON	The text format the websites use to send data: `{"chapter": 2, "text": "..."}`.
API	A website address that gives data (not a page) for programs to use. We call three of them.
Cache	A saved copy of an answer, so we do not ask the website again. Ours is the `cache` folder.
Hash	A one-way scramble of text. You can compute it from the password, but you cannot get the password back from it.
Salt	A random text mixed into the password before hashing, so two equal passwords give different hashes.
Layer	A group of classes with one job. A layer only calls the layer under it.
DAO	Data Access Object: a class whose only job is to find and save data.
Service	A class that holds the logic (the rules) of the system.
Controller	A class that answers one web address: reads the request, calls a service, builds the page.
Domain class	A class that describes a thing from our world: an issue, a verse, a user.
Constructor	The special method that creates an object: `new Ayah(1, 2, "Al-Baqarah", 232, "...")`.
Getter / setter	A method that reads (`getTitle()`) or changes (`setText(...)`) a private field.
private / public	`private` = only this class can use it. `public` = anyone can.
static	Belongs to the class, not to one object. You call it without `new`: `Html.escape(text)`.
null	"Nothing here". For example `findById` returns `null` when there is no such issue.
ArrayList	A list that grows. `ArrayList<Masala>` is a list of issues.
HashMap	A list of pairs "name → value". Used for sessions (token → user) and for form fields.

package / import	`package com.silsila.domain;` says which folder a class lives in. `import` lets us use a class from another folder.
implements HttpHandler	A promise that our class has a `handle` method, so the server can give it requests.
@Override	Says "I am replacing a method that the interface requires". It catches spelling mistakes.
try / catch	"Try this; if this kind of error happens, do the catch part instead of crashing."

3. The folders and how the program starts

```
Silsila-/
  pom.xml, README.md
  cache/                      saved answers from the websites (JSON files)
  docs/                       requirements, architecture, diagrams, these guides
  src/main/java/com/silsila/
    domain/        Masala  Ayah  Tafsir  Hadith  Qanun  MadhabPosition  Link  User
    dataaccess/    Database  MoreContent  MasalaDAO  AyahDAO  TafsirDAO  HadithDAO
                   QanunDAO  MadhabPositionDAO  LinkDAO  UserDAO  RemoteLoader  SimpleJson
    application/   MasalaService  MasalaChain  LinkService  AuthService  PasswordHasher
    presentation/  Main  Html  Request  SessionManager  SearchController
                   MasalaController  AuthController  ProposeController  ReviewController
```

A **package** is just the folder name inside Java. `com.silsila.domain` is the folder `com/silsila/domain`. We use one package per layer so the structure of the folders is the structure of the architecture.

What happens when you click Run on Main.java

Java starts with the method `public static void main(String[] args)` in `Main`. Here is every step, in order:

1. **`new Database()`**: creates the seven (eight with users) empty lists, then the constructor fills them by calling private methods: `addMasalas()`, `addAyat()`, `addTafsirs()`, `addHadiths()`, `addQanuns()`, `addPositions()`, `addLinks()`, and finally `MoreContent.addTo(this)` which adds the second batch of content. At this point verses and hadith only have placeholder texts like "(The Arabic text will be loaded...)".
2. **`new RemoteLoader(database).loadAll()`**: for every verse, tafsir and hadith it reads the saved file from `cache` or downloads it from the website, and puts the real text into the object using the setters (`setText`, `setTranslation`, `setGrade`, `setArabicText`).
3. **Create the DAOs**: `new MasalaDAO(database)` and so on. Each DAO is given the same `Database` object.
4. **Create the services**: `new MasalaService(...six DAOs...)`, `new LinkService(...)`, `new AuthService(userDAO)`.
5. **Create two demo accounts**: `authService.register(...)` hashes their passwords and saves them. (These two lines are at the end of the setup part of `Main.java`.)
6. **Create the web server**: `HttpServer.create(new InetSocketAddress(8080), 0)`.
7. **Register the controllers**: `server.createContext("/masala", new MasalaController(masalaService))` says "every request whose address starts with /masala goes to this object". We register six addresses: `/`, `/masala`, `/login`, `/logout`, `/propose`, `/review`. The server picks the *longest* matching address, so `/masala?id=4` goes to `MasalaController` and everything unknown goes to `/` (which shows a 404 page).
8. **`server.start()`**: from now on the server waits for browsers. `main` ends, but the server keeps the program alive.

> **Why Main creates everything:** a class never creates the class below it. `Main` builds all the objects and hands each object the ones it needs (through the constructor). This is called **constructor injection** (or "dependency injection"). It means each class only *uses* what it is given, so it is easy to understand and to replace.

4. How the browser and Java talk, and how HTML is joined with Java

4.1 A request, as the server sees it

When you type `http://localhost:8080/masala?id=4` the browser sends a text message like this:

```
GET /masala?id=4 HTTP/1.1
Host: localhost:8080
Cookie: session=3fa91c...        (only if you are logged in)
```

Java's `HttpServer` reads that text and creates an object called **`HttpExchange`**. This single object represents the whole conversation: it knows the method (`getRequestMethod()`), the address (`getRequestURI()`), the headers (`getRequestHeaders()`), the body of the request (`getRequestBody()`), and it is also how we write the answer (`sendResponseHeaders(...)` and `getResponseBody()`).

The server then calls the `handle(HttpExchange exchange)` method of the controller registered for that address. That is why every controller `implements HttpHandler`: the interface says "you must have a `handle` method".

4.2 The answer, as the browser receives it

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 4210

<!DOCTYPE html><html>... the whole page ...</html>
```

Our `Html.send(exchange, 200, html)` does exactly this: it turns the page text into bytes (UTF-8, so Arabic works), sets the `Content-Type` header so the browser knows it is HTML, sends the status line and the length, and writes the bytes.

4.3 How the HTML is joined with the Java (the most asked question)

There is **no template engine and no JavaScript**. The HTML is simply a Java `String` that we build with `+`. Example, from `MasalaController`:

```
String content = "<h1>" + Html.escape(masala.getTitle()) + "</h1>\n";
content = content + "<p>" + Html.escape(masala.getDescription()) + "</p>\n";
```

So the "joining" is: Java data (a title from an object) is put between pieces of HTML text. Loops make repeated parts: a `for` loop over the verses adds one `<div class="box">` per verse. When the string is finished it goes to `Html.page(title, content)`, which puts it inside the common layout (head, style, header menu, footer). Then `Html.send` sends it.

Piece	Where it is built
The part that changes (the issue, the list of results)	The controller, by String concatenation

The part that is the same on every page (head, CSS, header, footer)	`Html.page(...)`
The login/logout links at the top right	`Html.send(...)` replaces the marker `<!--NAV-->` by the right links, because `send` can ask `SessionManager` who is logged in
Styling	CSS inside the `<style>` tag written in `Html.page`
Phone-friendly layout	The `viewport` meta tag and the CSS grid with `auto-fit` in the madhab boxes

4.4 Why `Html.escape` exists

If a title contained `<script>alert(1)</script>` and we put it into the page as it is, the browser would *run* it. `escape` replaces the dangerous characters: `&` becomes `&`, `<` becomes `<`, `>` becomes `>`, `"` becomes `"`, `'` becomes `'`. The browser then shows them as plain characters. The rule in the whole project: **every piece of text that comes from data or from a user goes through `Html.escape`**. This protects against **XSS** (cross-site scripting). We tested it: a citation containing a script tag appeared as harmless text on the review page.

4.5 Reading what the browser sent

Where the data is	How we read it
In the address: `/search?q=wali`	`Request.getQueryParam(exchange, "q")`. It takes the part after `?`, splits it at `&` into pairs, splits each pair at `=`, finds the name, and decodes it with `URLDecoder` (a space travels as `+` or `%20`).
In the body of a form (POST)	`Request.getForm(exchange)`. It reads all the bytes of the body (for example `username=ali&password=xyz`), splits at `&` and `=`, decodes each part, and returns a `HashMap` of name → value. The body can only be read **once** per request, so we call it once.
In a cookie	`SessionManager.getUser(exchange)` (see section 9).

4.6 GET and POST in our forms

A page with a form is shown by a **GET** request (the server sends the form). When you press the button the browser sends a **POST** request to the address in `action="..."` with the fields in the body. The same controller handles both: it looks at `exchange.getRequestMethod()` and chooses what to do. After a successful POST we either show a "thank you" page or **redirect** (status 303 and a `Location` header): the browser then makes a new GET to the new address. This stops the problem of the form being sent twice when you refresh.

5. The domain classes

These classes only **hold data**: private fields, a constructor, getters, and a few setters. They have almost no logic. Every layer uses them to pass information around. Records are connected by **ids** (like student numbers), not by holding each other.

Class	What it is	Fields and notable methods
Masala	One fiqh issue	id, title, category ("NIKAH"/"TALAQ"), chapter, description, keywords (words used by the search)
Ayah	A verse. We only type its place; the texts are loaded.	id, surahNumber, surahName, ayahNumber, text (Arabic), translation (English). `getReference()` returns "Surah Al-Baqarah 2:232". Setters: `setText`, `setTranslation`
Tafsir	An explanation of a verse by a scholar	id, **ayahId** (which verse), sourceName, edition (the website's book name), text (loaded)
Hadith	A hadith. We only type which one.	id, collection (website name, e.g. "bukhari"), collectionName (shown), **number** (the usual citation), **apiNumber** (number on the website, sometimes different, e.g. Muslim 1472 = 3673 on the website), text, arabicText, grade. `getReference()` gives "Sahih Muslim no. 1472"
Qanun	A modern law article	id, lawName, article, description. `getReference()`
MadhabPosition	What one school says about one issue	id, **masalaId**, madhab, ruling, evidence, illah (reasoning), citation (the book)
Link	The connection "this issue is supported by this verse/hadith/law". The heart of the project.	id, **masalaId**, type ("AYAH"/"HADITH"/"QANUN"), **targetId** (id of the verse/hadith/law), citation, status ("PENDING"/"APPROVED"/"REJECTED"), proposedBy. `isApproved()`, `isPending()`, `setStatus()`
User	A person with an account	id, username, passwordHash (never the password), role ("CONTRIBUTOR"/"REVIEWER"). `isReviewer()`

> **How ids connect things.** `Tafsir.ayahId` says which verse it explains. `MadhabPosition.masalaId` says which issue it belongs to. A `Link` has `masalaId`, `type` and `targetId`: for example *masalaId 4, type AYAH, targetId 3* means "issue 4 is supported by verse number 3". The `type` tells us *which list* the `targetId` belongs to.

Why Strings for category, type, status and role? It keeps the code simple for a beginner. The weakness: a spelling mistake ("APROVED") is not caught by the compiler. A more advanced design uses an `enum`. You can say this honestly.

Why does `Hadith` have two numbers? The website that gives the hadith numbers Sahih Muslim in its own way. Students expect the usual citation (1472) but to download we need the website's number (3673). So we store both.

6. The Database and the content files

6.1 Database

It is not a real database. It is a class with eight public lists: `masalas`, `ayat`, `tafsirs`, `hadiths`, `qanuns`, `positions`, `links`, `users`. The data is created when the program starts and is lost when it stops (so approvals and new proposals do not survive a restart). The lists are `public` only so the DAOs can reach them; no other layer should touch them.

6.2 MoreContent

The second batch of content (issues 7 to 16, positions for issues 2, 5, 6, more verses, hadith, laws and links). It is a separate class so the first file stays small. `Database`'s constructor ends with `MoreContent.addTo(this)`, and `addTo` calls its own private methods. It has two helper methods, `position(...)` and `link(...)`, which give each new record the next id automatically (`list.size() + 1`) so nobody has to count ids by hand. All of this content is **draft data** written from memory and must be verified.

7. The DAO classes

A DAO is the only kind of class that knows **where the data lives and how to find it**. The layers above never loop over the lists themselves. Every DAO receives the `Database` in its constructor and keeps it in a private field. Because only DAOs touch the data, we could replace the lists by a real database later and change only the DAOs.

DAO	Methods and how they work
MasalaDAO	`findAll()` returns the list. `findById(id)` loops until it finds the id, else returns `null`. `findByCategory(c)` returns the issues whose category `equals` c. `searchByKeyword(text)`: lowercases the text, cuts it into words at spaces, and for each issue builds a text `(title + " " + keywords).toLowerCase()`; the issue is a result only if **all** the words are inside that text (`contains`). An empty search gives a single empty word, which every text contains, so it returns all issues.
AyahDAO	`findAll()`, `findById(id)`, and `findApprovedByMasala(masalaId)`: loops over **links**; for every link whose masalaId matches, whose type is "AYAH" **and** which `isApproved()`, it finds the verse with `link.getTargetId()` and adds it to the result.
HadithDAO	Same three methods for hadith (type "HADITH").
QanunDAO	Same three methods for laws (type "QANUN").
TafsirDAO	`findByAyah(ayahId)`: the tafsir entries whose `ayahId` matches. (Tafsir is not behind links: any tafsir of an approved verse is shown.)
MadhabPositionDAO	`findByMasala(masalaId)`: the positions of one issue.
LinkDAO	Used by the curation workflow, so it sees links of **every** status: `findAll()`, `findById(id)`, `findPending()` (status PENDING), `nextId()` (biggest id + 1), `add(link)`.

UserDAO	`findByUsername(name)` (or `null`) and `add(username, passwordHash, role)` which gives the new user the next id.

> **The most important rule, enforced here:** only APPROVED links reach students. The three `findApprovedByMasala` methods skip every link that is not approved. Even if a page forgot the rule, a pending or rejected link could not reach a student, because the page never receives it.

8. The services (the logic)

8.1 MasalaService

It is given the six DAOs in its constructor. Its methods:

- `getAllMasalas()`: asks `MasalaDAO.findAll()`.
- `searchByKeyword(keyword)`: replaces `null` by an empty text (so a missing search does not crash), then asks the DAO.
- `getChapters(category)`: loops over the issues of the category and collects the chapter names **without repeating** a name (`if (!chapters.contains(...))`).
- `getMasalasInChapter(category, chapter)`: the issues of that category whose chapter equals the name. These two build the Browse tree: Book → Chapter → Issue.
- `getChain(masalaId)`: the main method. It finds the issue (if none, returns `null`), asks `AyahDAO` for its approved verses, then for **each verse** asks `TafsirDAO` for its tafsir and adds them all (`addAll`), then asks for approved hadith, positions and approved laws, and packs everything into one `MasalaChain`.

8.2 MasalaChain

A "parcel": it holds the issue, its verses, tafsirs, hadiths, positions and laws, with getters. `getTafsirsOf(ayahId)` picks the tafsir of one verse (so the page can show each verse followed by its own tafsir). The page receives one object instead of six lists.

8.3 AuthService (accounts)

- `register(username, password, role)`: refuses (returns `false`) if the username is empty, the password is shorter than 6 characters, the role is not CONTRIBUTOR or REVIEWER, or the username already exists. Otherwise it saves the user with `PasswordHasher.hash(password)`. The real password is never stored.
- `login(username, password)`: finds the user; if none, returns `null`. Otherwise `PasswordHasher.matches(password, user.getPasswordHash())`; if it matches returns the user, else `null`.

8.4 PasswordHasher (how password hashing works, step by step)

Goal: if someone steals our data, they must not be able to read the passwords (requirement NFR-3).

Saving a password: `hash(password)`

1. Make a random **salt**: 8 random bytes from `SecureRandom` (a safe random generator), written as 16 hex characters (digits 0-9 and letters a-f), for example `3fa91c07b2d4e1a8`.
2. Put the salt in front of the password: `salt + password`.
3. Give that text to `MessageDigest.getInstance("SHA-256")`. SHA-256 turns any text into 32 bytes of scrambled data. The same input always gives the same output, but you cannot go backwards. A tiny change in the input changes the whole output.
4. Write the 32 bytes as 64 hex characters (`toHex` uses `String.format("%02x", b)` for each byte).
5. Store `salt + ":" + hash`, for example `3fa91c07b2d4e1a8:9b71d2...`. That is what is saved in `User.passwordHash`.

Checking a login: `matches(typedPassword, stored)`

1. Cut the stored text at the colon: the first part is the salt, the second is the expected hash.

2. Compute `SHA-256(salt + typedPassword)` again with the **same salt**.
3. If the result equals the expected hash, the password is right. We never needed the real password from the database.

> **Why a salt?** Without it, two users with the same password would have the same hash, and an attacker could use a ready-made table of hashes of common passwords. With a random salt each hash is different, so such tables do not work.

> **Be honest in the viva:** SHA-256 is very fast, and a fast hash is not ideal for passwords because an attacker can try many guesses per second. Real systems use slow, purpose-made algorithms such as **bcrypt**, **scrypt** or **PBKDF2**. For a course project, salted SHA-256 shows the idea correctly, and it needs no extra library. This is a known improvement.

Why `static` methods? The hasher has no data of its own, so it does not need an object: `PasswordHasher.hash(...)`.

8.5 LinkService (the curation workflow)

Its methods that can fail return a **text**: `""` means success; anything else is the message to show to the person. (We chose this instead of exceptions because it is simpler to read.)

- `propose(user, masalaId, type, targetId, citation)` checks in order: the user is logged in; the issue exists; **the citation is not empty (NFR-4: no link without a citation)**; the citation is at most 300 characters; the target (verse/hadith/law) really exists; the same link does not already exist (unless the old one was rejected). If all is fine it creates a `Link` with status **PENDING**, sets `proposedBy` to the username, and saves it with `LinkDAO.add`.
- `decide(reviewer, linkId, decision)`: the person must be a reviewer (`isReviewer()`); the link must exist and be pending; `"approve"` sets status APPROVED, `"reject"` sets REJECTED.
- `getPendingLinks()`, `describeTarget(link)` (turns "AYAH:3" into "Surah Al-Baqarah 2:229"), `getMasalaTitle(id)`, `getAllAyat/Hadiths/Qanuns()` (to fill the drop-down list of the proposal form).

> **Where the rules live:** the "citation required" rule and the "only reviewers can approve" rule are in the **service**, not only in the page. Even if someone changes the form in their browser or sends a hand-made request, the rule still holds. The page just hides the buttons for convenience; the service is the real lock.

9. The presentation classes

9.1 Main

Explained in section 3: builds everything, registers the six addresses, starts the server.

9.2 Html

- `escape(text)`: protects against XSS (4.4). Returns `""` for `null`.
- `page(title, content)`: builds the complete page: doctype, head (UTF-8, viewport, title, CSS), a header with the name and the `<!--NAV-->` marker, the content inside `<main>`, and a footer.
- `navFor(user)` (private): "Log in" if nobody is logged in; otherwise the username and role, a "Review links" link for reviewers, and "Log out".
- `send(exchange, status, html)`: replaces the marker with `navFor(SessionManager.getUser(exchange))`, writes the page. This is why none of the other controllers needed to know about the menu.
- `redirect(exchange, address)`: status 303 with a `Location` header, no body.

9.3 Request

`getQueryParam` and `getForm` (explained in 4.5).

9.4 SessionManager (how "being logged in" works)

The web forgets you between requests: each request arrives alone. To remember who you are:

1. **Login succeeds.** `SessionManager.login` creates a **token**: 16 random bytes written as 32 hex characters (nobody can guess it). It stores `token → user` in a static `HashMap` called `sessions`.
2. **Give the token to the browser** with a response header: `Set-Cookie: session=TOKEN; Path=/; HttpOnly; SameSite=Strict`.
3. **The browser sends it back** on every later request in the header `Cookie: session=TOKEN`.
4. **Who is asking?** `getUser(exchange)` reads the `Cookie` header, cuts it at `;` and `=` to find `session`, and looks the token up in the map. If it is not there, the result is `null` = nobody is logged in.
5. **Logout.** `logout` removes the token from the map and sends the cookie again with `Max-Age=0`, which tells the browser to delete it.

Cookie flag	Meaning
`Path=/`	The cookie is sent for every address of the site.
`HttpOnly`	JavaScript in the page cannot read the cookie, so an XSS attack cannot steal it.
`SameSite=Strict`	The browser does not send the cookie when the request comes from another website. This helps against CSRF (a bad site making your browser send a forged form).

Why is a plain `HashMap` safe? The JDK server, with its default settings, handles requests one after another on one thread, so two requests do not change the map at the same time. A bigger system would use a thread-safe map.

9.5 SearchController (addresses `/` and `/search`)

- `/` → `showHome`: builds the title, the search form, and the Browse tree. For each book ("NIKAH", "TALAQ") it asks `getChapters`, and for each chapter `getMasalasInChapter`, and makes a link `/masala?id=N` for every issue.
- `/search` → `showResults`: reads `q`, records `System.currentTimeMillis()` before and after the search to show "N issue(s) found in X ms" (requirement NFR-1: under 2 seconds), shows each result with its book, chapter and description, or a "no issue matches" message.
- Any other address → a 404 page (this controller is registered on `/`, so it receives everything that no other controller took).
- The private helpers `searchForm(query)` (the search box, with the typed text put back into it, escaped) and `categoryName(...)` ("NIKAH" → "Nikah (Marriage)").

9.6 MasalaController (address `/masala?id=N`)

1. Reads `id` and converts it with `Integer.parseInt` inside `try/catch`. If it is not a number (`?id=abc`) we use -1, which no issue has.
2. `masalaService.getChain(id)`. If the result is `null` (unknown issue) it sends a 404 page and stops with `return;`.
3. Builds four sections: **1. Qur'an and tafsir** (for each verse: its reference, the Arabic text in a `dir="rtl"` paragraph so it reads right to left, the English translation, and each tafsir inside `<details>` which opens when clicked because tafsir texts are long); **2. Hadith** (reference, Arabic, English, and the grade if it is not empty); **3. Positions of the madhabs** (a CSS grid of boxes: ruling, evidence, reasoning, source; if none exist, a "not added yet" message); **4. Qanun**.
4. At the end: if nobody is logged in, a "Log in to propose..." message; otherwise a link `/propose?id=N`.
5. Every section has a friendly message when it is empty, so nothing crashes or looks broken.

9.7 AuthController (addresses `/login` and `/logout`)

- **GET /login**: shows the form (username, password).
- **POST /login**: `Request.getForm`, then `authService.login(username, password)`. If `null`: shows the form again with "Wrong username or password." (the **same message** whether the username or the password was wrong, so nobody can find out which usernames exist). If a user: `SessionManager.login(exchange, user)` (sets the cookie) and redirects to `/`.
- **/logout**: `SessionManager.logout` and redirect to `/`. The same class is registered on both addresses.

9.8 ProposeController (address `/propose`)

- First: `SessionManager.getUser`. If `null` → redirect to `/login`.
- **GET** `/propose?id=4`: shows the form: a hidden field with the issue id, a drop-down (`<select>`) listing every verse, hadith and law (each option's value looks like `AYAH:3`: the type and the id), and a text box for the citation. Unknown issue → 404.
- **POST**: reads the form, splits the target at the colon into type and id, calls `linkService.propose(...)`. If it returns an error text the form is shown again with a red message and the citation kept; otherwise a "Thank you, it will appear after a reviewer approves" page.

9.9 ReviewController (address `/review`)

- Not logged in → redirect to `/login`. Logged in but not a reviewer → a **403** "Not allowed" page.

- **GET**: lists the pending links: issue title (as a link), the proposed evidence (`describeTarget`), the citation, who proposed it, and a small form with two buttons (`name="decision"` with value `approve` or `reject`) and a hidden `linkId`.
- **POST**: `linkService.decide(user, linkId, decision)`, then shows the list again with a green "approved/rejected" message or a red error.

10. Loading verses, tafsir and hadith from websites

We do not type the Qur'an, tafsir or hadith: a typing mistake in a verse would be a serious error. Instead we store **where** each item is (surah and ayah number, book and hadith number) and download the text from free public data on GitHub (through the jsDelivr CDN):

Content	Source (edition)
Arabic verse	fawazahmed0/quran-api, `ara-quranuthmanihaf/{surah}/{ayah}.json`
English translation	same project, `eng-ummmuhammad`
Tafsir Ibn Kathir (English)	spa5k/tafsir_api, `en-tafisr-ibn-kathir/{surah}/{ayah}.json`
Hadith (Arabic, English, grades)	fawazahmed0/hadith-api, `ara-` and `eng-` editions

10.1 RemoteLoader

- `loadAll()` calls `loadAyat()`, `loadTafsirs()`, `loadHadiths()`. Each loops over the objects in the `Database`, builds the website address, gets the JSON text, and puts the values into the object with the setters.
- `get(url, cacheName)` is the heart: (1) if the file `cache/cacheName` exists, read and return it; (2) if we already learned that the internet is down (`offline` flag), return `null` at once; (3) otherwise ask the website with `HttpClient` (a 20 second timeout, up to **three tries**, one second pause between them, because the CDN sometimes answers a random 403); if the answer is 200, save it in `cache` and return it. If the website never answers, set `offline = true` so we do not wait for every other file.
- If a text cannot be loaded, the page shows "(Could not load the text. Check the internet connection and restart.)" instead of crashing.
- `readGrades(json)`: the hadith file has a list `"grades": [{"name": "Al-Albani", "grade": "Sahih"}, ...]`. We loop through it and build one line, for example "Sahih (Al-Albani), Sahih (Zubair Ali Zai)". Sahih al-Bukhari and Sahih Muslim have no grades in this source, so nothing is shown for them.

10.2 SimpleJson

A tiny reader instead of a library (so there is nothing to install). `getString(json, "text")` finds the first `"text"`, skips the colon and spaces, expects an opening quote, then copies characters until the closing quote. When it sees a backslash it understands `\n` (new line), `\t`, `\uXXXX`-style Unicode codes, and `\"` (a quote inside the text). It returns `null` if the key is missing or the value is not text.

10.3 The cache

The `cache` folder contains the saved JSON answers (names like `ara-2-232.json`, `tafsir-en-tafisr-ibn-kathir-2-232.json`, `eng-bukhari-5087.json`). It makes startup fast (a few seconds instead of about a minute) and lets the project work without internet. We commit it to GitHub so everyone who clones the project has the texts.

11. Full walk-throughs

A. A student opens the home page

1. Browser: `GET /`.
2. The server finds the context `/` → `SearchController.handle`. The path equals "/" → `showHome`.
3. `showHome` asks `MasalaService.getChapters("NIKAH")` → `MasalaDAO.findByCategory` → the lists in `Database`. Same for "TALAQ".
4. It builds the HTML, wraps it in `Html.page`, and `Html.send` adds the login links and writes the response (200).

B. A student searches for "wali"

1. The form (`method="get" action="/search"`) makes the browser request `GET /search?q=wali`.
2. `SearchController.showResults`: `Request.getQueryParam(exchange, "q")` → "wali".
3. `MasalaService.searchByKeyword("wali")` → `MasalaDAO.searchByKeyword`: lowercase, split into words, check each issue's title + keywords.
4. The page shows "1 issue(s) found in 0 ms" and the result box with a link to `/masala?id=1`.

C. A student opens an issue

1. `GET /masala?id=4` → `MasalaController`.
2. `MasalaService.getChain(4)` → `MasalaDAO.findById(4)`; `AyahDAO.findApprovedByMasala(4)` (looks in `links` for approved AYAH links of issue 4); for each verse `TafsirDAO.findByAyah`; `HadithDAO.findApprovedByMasala(4)`; `MadhabPositionDAO.findByMasala(4)`; `QanunDAO.findApprovedByMasala(4)`.
3. The service returns a `MasalaChain`; the controller turns it into HTML (four sections) and sends it.

D. A contributor logs in

1. `GET /login` shows the form. The user types the username and password and presses the button: `POST /login` with the body `username=...&password=...`.
2. `AuthController.checkLogin` → `Request.getForm` → `AuthService.login` → `UserDAO.findByUsername` → `PasswordHasher.matches` (re-hash with the stored salt and compare).
3. Success: `SessionManager.login` stores token → user and sends `Set-Cookie`; `Html.redirect` to "/" (303).
4. The browser follows the redirect with the cookie; `Html.send` asks `SessionManager.getUser` and shows the name and "Log out" in the header.

E. A contributor proposes a link

1. On an issue page the contributor clicks "Propose a link for this issue" → `GET /propose?id=2`. `ProposeController` checks the session, then shows the form.
2. The contributor picks a verse and writes a citation, presses "Send for review": `POST /propose` with `masalaId`, `target=AYAH:3`, `citation=...`.
3. `ProposeController` splits `AYAH:3`, calls `LinkService.propose(...)`. The service checks the rules and saves a `Link` with status PENDING through `LinkDAO.add`.

4. The "Thank you" page is shown. The issue page does **not** show the new link yet, because `AyahDAO.findApprovedByMasala` only returns APPROVED links.

F. A reviewer approves it, and students see it

1. The reviewer logs in (same as D). The header now shows "Review links".
2. `GET /review` → `ReviewController` checks `user.isReviewer()`, lists `LinkService.getPendingLinks()`.
3. Click "Approve": `POST /review` with `linkId=10&decision=approve` → `LinkService.decide` → `link.setStatus("APPROVED")`.
4. Now `AyahDAO.findApprovedByMasala` finds that link, and the verse appears on the issue page for everyone.
5. If the reviewer clicks "Reject", the status becomes REJECTED, it never appears, and the same link can be proposed again later.

12. Security: what protects what

Threat	Protection in our code	Where
Someone injects a script through a title, search text or citation (XSS)	Every dynamic text goes through `Html.escape`	All controllers
Passwords stolen from the data	Only salted SHA-256 hashes are stored	`PasswordHasher`, `AuthService`
Session cookie stolen by a script	`HttpOnly`	`SessionManager`
Forged form from another website (CSRF)	`SameSite=Strict` on the cookie (a partial protection; no CSRF tokens)	`SessionManager`
A student or contributor approves links	Role check in `decide` and in `ReviewController` (403)	`LinkService`, `ReviewController`
A link without a source	Empty citation refused (NFR-4)	`LinkService.propose`
Unapproved content reaches students	The DAOs return only APPROVED links	`AyahDAO`, `HadithDAO`, `QanunDAO`
Bad input crashes the server	`try/catch` around number conversions; null checks; 404 page	Controllers, `MasalaService`
Finding which usernames exist	The same login error message for both cases	`AuthController`
Guessing the session token	16 random bytes from `SecureRandom`	`SessionManager`

> **Not protected (say so before the doctor finds it):** no HTTPS (passwords travel as plain text on a real network; on localhost they do not leave your computer); no limit on wrong password attempts; no CSRF tokens; the demo accounts are written in `Main.java`, and the repository is public, so anyone can read those demo passwords; a reviewer can approve their own proposal; sessions and data are lost on restart.

13. Why we designed it this way

Decision	Reason
Four layers	Each layer has one job, so it is easy to understand, test and change. For example, changing where data comes from only touches the DAOs.
A layer only calls the layer below it	Controllers never loop over data; DAOs never build HTML. Mixing them would make changes break many places.

No framework	The project is for learning: using the JDK's own server and plain Java shows what frameworks hide. It also needs nothing installed except Java.
HTML built with Strings	Simple to follow for a beginner; no template language to learn.
Data in memory	Fast to build and easy to demo; the cost is that nothing is saved after a restart.
Verses and hadith loaded from websites	Nobody can mistype a verse; the source is public and citable; the cache makes it work offline.
One `Link` class with a status	One simple class for the workflow PENDING → APPROVED/REJECTED. (Our first design diagram had separate AyahLink, HadithLink and QanunLink classes and a User hierarchy; we simplified, and this is stated openly.)
Services return a message text, not exceptions	Easier for a beginner to read; the controller just shows the message.
Constructor injection (Main wires everything)	Classes do not create their own helpers, so any part can be replaced.
Redirect after POST	Prevents the form being sent twice when the page is refreshed.

14. Limits (be honest about these)

- **Data is in memory.** Proposals, approvals and logins disappear when the program stops.
- **Content is draft.** The madhab positions, book references and law articles were written from memory and must be checked against the original books and laws. All citations say "(add page)".
- **Two hadith numbering systems** exist (usual citation versus the website's). We handle this for Muslim by keeping both numbers.
- **Password hashing** uses salted SHA-256, which is fast; bcrypt/PBKDF2 would be better.
- **No automated tests** in the project (we tested by running the pages and the flows by hand).
- **No HTTPS, no rate limiting, no CSRF tokens**, demo accounts in the code.
- **Strings instead of enums** for category, type, status and role.
- **A reviewer can approve their own proposal**; there is no "second person" rule.
- **The search** is a simple "contains" on title and keywords (no ranking, no spelling help).
- **Tafsir is not behind links:** any tafsir of an approved verse is shown.

How to improve (good to say if asked): a real database (H2 or PostgreSQL) in the DAOs; bcrypt; HTTPS; CSRF tokens; enums; automated tests (JUnit) for the services and DAOs; a page to create accounts; a rule that reviewers cannot approve their own proposals; audit log of who approved what.

15. Questions the doctor may ask, with answers

Q1. What is the architecture of your system?

A layered architecture with four layers: Presentation (web server, controllers, HTML), Application (services with the logic), Data access (DAO classes) and Database (lists in memory), plus shared domain classes. Each layer only calls the one below it.

Q2. Why layers?

Separation of concerns: each layer has one job, so we can change one without breaking the others, and each is easier to understand and test.

Q3. What does a DAO do and why do you need it?

It is the only class that knows where the data lives and how to find it. The services ask it for data. If we move from lists to a real database, only the DAOs change. It also enforces "only approved links reach students".

Q4. What does the service layer do that the DAO does not?

The DAO only fetches or stores. The service holds the rules and combines results: for example `getChain` collects the verses, their tafsir, the hadith, positions and laws; `propose` and `decide` enforce the rules about citations and roles.

Q5. How does the browser talk to your Java code?

Through HTTP. The JDK `HttpServer` listens on port 8080, turns each request into an `HttpExchange`, and calls the `handle` method of the controller registered for that address. The controller builds an HTML text and writes it back with a status code.

Q6. How is HTML joined with Java?

The HTML is a Java String built with concatenation. Controllers add data between pieces of HTML and loops for repeated parts; `Html.page` wraps it in the common layout; `Html.send` writes it as UTF-8. There is no template engine and no JavaScript.

Q7. What is the difference between GET and POST, and where do you use each?

GET asks for a page, with data in the address (search, issue page, forms to show). POST sends data to change something, with the data in the body (login, propose, approve/reject).

Q8. Why do you redirect after a successful login?

So that refreshing the page does not send the form again (the browser does a new GET), and to take the user to a normal page. We use status 303.

Q9. How does login work?

The form POSTs the username and password. `AuthService.login` finds the user and checks the password with `PasswordHasher.matches`. On success `SessionManager` creates a random token, stores token → user, and sends it as a cookie. The browser returns the cookie on every request, so the server knows who is asking.

Q10. How do you store passwords?

We never store the password. We store `salt:hash`, where hash is SHA-256 of salt + password. To check a login we hash the typed password with the same salt and compare.

Q11. What is a salt and why use it?

A random text added to the password before hashing. Equal passwords then give different hashes, so ready-made tables of common-password hashes are useless.

Q12. Is SHA-256 good enough for passwords?

It shows the correct idea (one-way, salted), but it is fast, which helps attackers guess. A real system should use bcrypt, scrypt or PBKDF2. That is a known improvement.

Q13. What is a session and how do you keep it?

The server's memory that a cookie token belongs to a user. We keep a static `HashMap` from token to `User` in `SessionManager`. It is lost on restart, which logs everybody out.

Q14. What are HttpOnly and SameSite on your cookie?

HttpOnly: page scripts cannot read it. SameSite=Strict: the browser does not send it with requests started by other websites, which helps against CSRF.

Q15. What is XSS and how do you prevent it?

Injecting a script through text that the site then shows to other people. We escape every dynamic text with `Html.escape`, which replaces &, <, >, quotes, so the browser shows them as text.

Q16. What is CSRF and are you protected?

A malicious site makes your browser send a request to our site using your cookie. We have partial protection from `SameSite=Strict`. We do not use CSRF tokens; that would be the next step.

Q17. How do you make sure unapproved links never reach students?

The DAOs (`findApprovedByMasala`) only return items whose link has the status APPROVED. The page code never receives the others.

Q18. How do you enforce that only reviewers can approve?

In two places: `ReviewController` returns 403 for non-reviewers, and `LinkService.decide` checks `isReviewer()` again. The service check is the real lock because it also protects against hand-made requests.

Q19. How do you enforce that every link has a citation?

`LinkService.propose` refuses an empty citation (requirement NFR-4), whatever the browser sent. The form also marks the field as required in its text.

Q20. Where does the text of verses, tafsir and hadith come from?

From public JSON data on GitHub served by jsDelivr. `RemoteLoader` downloads it with `HttpClient` and fills the objects through setters. Every answer is saved in the `cache` folder.

Q21. What if the website is down?

If the file is in the cache we use it. If not, the loader retries up to three times, then the page shows a clear "could not load" message. If the internet is missing entirely the loader stops asking after the first failure, so startup is not slow.

Q22. Why did you write your own JSON reader?

To avoid installing a library and to keep the code simple. `SimpleJson` reads one text value, including escapes. A real project would use Gson or Jackson.

Q23. How does the search work?

The typed text is lowercased and split into words. An issue matches if every word appears in its title plus keywords (`contains`). An empty search matches everything. We measure and show the time in milliseconds (NFR-1).

Q24. How does the Browse tree work?

`getChapters` collects the distinct chapter names of a book; `getMasalasInChapter` gives the issues of a chapter. The home page loops over both to print Book → Chapter → Issue.

Q25. How are the four madhabs shown side by side?

A CSS grid (`display:grid` with `repeat(auto-fit, minmax(200px, 1fr))`) puts one box per madhab. On a phone the boxes stack automatically (NFR-2).

Q26. Why is Arabic shown correctly?

The page is UTF-8 (the `meta charset` and the response header), and the Arabic paragraphs have `dir="rtl"` so they read right to left.

Q27. What does `null` mean in `findById` and how do you handle it?

It means "no such record". The caller checks for it: for example `getChain` returns `null` for an unknown issue and the controller then sends a 404 page.

Q28. Why do you use `.equals` and not `==` for Strings?

`==` compares whether two variables point to the same object; `.equals` compares the text. We want to compare the text.

Q29. What is constructor injection?

A class receives the objects it needs through its constructor instead of creating them. `Main` creates the Database, DAOs and services and passes them down. This keeps classes independent and replaceable.

Q30. What is the difference between a private field with a getter and a public field?

A private field can only be changed through methods that we control (encapsulation). The only public fields are the lists in `Database`, which exist so the DAOs can reach them.

Q31. Why is `PasswordHasher` made of static methods?

It has no data of its own, so no object is needed. You call it directly: `PasswordHasher.hash(password)`.

Q32. What happens when two users use the site at the same time?

The JDK server with default settings handles requests one after another on one thread, so our shared lists and the session map are not changed at the same time. A bigger system would need thread-safe collections or a real database.

Q33. What requirements did you meet?

FR-1 chain of evidence, FR-2 four madhabs side by side, FR-3 search, FR-4 browse, FR-5 citation on every item, FR-6 contributors propose links, FR-7 reviewers approve or reject, NFR-1 search time shown, NFR-2 works on phones, NFR-3 hashed passwords, NFR-4 no link without a citation.

Q34. What are the limits of the project?

See section 14: in-memory data, draft content to be verified, no HTTPS or rate limiting or CSRF tokens, fast hash, no automated tests, strings instead of enums.

Q35. What would you improve with more time?

A real database, bcrypt, HTTPS, CSRF tokens, enums, JUnit tests, account creation page, "reviewer cannot approve own proposal", an audit log, and verified content with exact page numbers.

Q36. Why `try/catch` around `Integer.parseInt`?

If someone writes `/masala?id=abc`, `parseInt` throws a `NumberFormatException`. We catch it and use -1, so the user gets a "not found" page and not a crash.

Q37. Why do you check the link already exists when proposing?

To avoid duplicates in the review list. A previously rejected link may be proposed again.

Q38. Why does the Hadith class have both `number` and `apiNumber`?

The usual citation and the website's numbering differ for some books (Sahih Muslim 1472 is number 3673 on the website). Students see the first; the loader uses the second.

Q39. How do the controllers find out who is logged in?

Through `SessionManager.getUser(exchange)`. `Html.send` also uses it to draw the right links in the header.

Q40. What is `HttpExchange`?

The JDK object that represents one request and its response: method, address, headers, request body, and the way to send the answer.

16. One-line cheat sheet of every class

Class	One line
Masala	An issue: title, book, chapter, description, keywords.
Ayah	A verse: place, Arabic text, translation (loaded).
Tafsir	An explanation of one verse (loaded text).
Hadith	A hadith: book, number, website number, texts, grade.
Qanun	A law article.
MadhabPosition	One school's view on one issue.
Link	Issue ↔ evidence with a citation and a status.
User	Account: username, password hash, role.
Database	The in-memory lists, filled at start-up.
MoreContent	The second batch of issues, positions, verses, hadith, laws, links.
MasalaDAO	Find/search issues.
AyahDAO / HadithDAO / QanunDAO	Find items, and find the APPROVED ones of an issue.
TafsirDAO	Tafsir of a verse.
MadhabPositionDAO	Positions of an issue.
LinkDAO	Find, add and list pending links (all statuses).
UserDAO	Find or add a user.
RemoteLoader	Downloads texts from websites, with cache and retries.
SimpleJson	Reads one text value from JSON.
MasalaService	Search, browse, and build the whole chain of an issue.
MasalaChain	The parcel holding the whole chain.
LinkService	Propose and approve/reject with the rules.
AuthService	Register and log in.
PasswordHasher	Salted SHA-256 hashing and checking.
Main	Builds and connects everything, starts the server.
Html	Escape, page layout, send, redirect, menu.
Request	Read address values and form fields.
SessionManager	Cookie token → logged-in user.
SearchController	Home page, search results, 404.

MasalaController	The issue page with the whole chain.
AuthController	Login form, login check, logout.
ProposeController	Propose-a-link form and saving.
ReviewController	Reviewer list and approve/reject.